Системы команд процессора. Регистры процессора: сущность, назначение, типы. Параллелизм вычислений.

Системы команд процессора.

Системы команд процессора (ISA)

Система команд определяет, какие инструкции умеет выполнять процессор, с какими типами данных он работает и как обращается к памяти.

Системы команд процессора (ISA)

Любая команда в памяти хранится как набор нулей и единиц. Логически она делится на две части:

Код операции (Opcode)

Поле операндов

Основные группы команд

- Пересылка данных
- Арифметические
- Логические
- Управление последовательностью (ветвление)

ADD Eb Gb *00*	**ADD** Ev Gv *01*	**ADD** Gb Eb *02*	**ADD** Gv Ev *03*	**ADD** AL Ib *04*	**ADD** eAX Iv *05*	**PUSH** ES *06*	**POP** ES *07*	**OR** Eb Gb *08*	**OR** Ev Gv *09*	**OR** Gb Eb *0A*	**OR** Gv Ev *0B*	**OR** AL Ib *0C*	**OR** eAX Iv *0D*	**PUSH** CS *0E*	**TWOBYTE** *0F*
ADC Eb Gb *10*	**ADC** Ev Gv *11*	**ADC** Gb Eb *12*	**ADC** Gv Ev *13*	**ADC** AL Ib *14*	**ADC** eAX Iv *15*	**PUSH** SS *16*	**POP** SS *17*	**SBB** Eb Gb *18*	**SBB** Ev Gv *19*	**SBB** Gb Eb *1A*	**SBB** Gv Ev *1B*	**SBB** AL Ib *1C*	**SBB** eAX Iv *1D*	**PUSH** DS *1E*	**POP** DS *1F*
AND Eb Gb *20*	**AND** Ev Gv *21*	**AND** Gb Eb *22*	**AND** Gv Ev *23*	**AND** AL Ib *24*	**AND** eAX Iv *25*	**ES:** *26*	**DAA** *27*	**SUB** Eb Gb *28*	**SUB** Ev Gv *29*	**SUB** Gb Eb *2A*	**SUB** Gv Ev *2B*	**SUB** AL Ib *2C*	**SUB** eAX Iv *2D*	**CS:** *2E*	**DAS** *2F*
XOR Eb Gb *30*	**XOR** Ev Gv *31*	**XOR** Gb Eb *32*	**XOR** Gv Ev *33*	**XOR** AL Ib *34*	**XOR** eAX Iv *35*	**SS:** *36*	**AAA** *37*	**CMP** Eb Gb *38*	**CMP** Ev Gv *39*	**CMP** Gb Eb *3A*	**CMP** Gv Ev *3B*	**CMP** AL Ib *3C*	**CMP** eAX Iv *3D*	**DS:** *3E*	**AAS** *3F*
INC eAX *40*	**INC** eCX *41*	**INC** eDX *42*	**INC** eBX *43*	**INC** eSP *44*	**INC** eBP *45*	**INC** eSI *46*	**INC** eDI *47*	**DEC** eAX *48*	**DEC** eCX *49*	**DEC** eDX *4A*	**DEC** eBX *4B*	**DEC** eSP *4C*	**DEC** eBP *4D*	**DEC** eSI *4E*	**DEC** eDI *4F*
PUSH eAX *50*	**PUSH** eCX *51*	**PUSH** eDX *52*	**PUSH** eBX *53*	**PUSH** eSP *54*	**PUSH** eBP *55*	**PUSH** eSI *56*	**PUSH** eDI *57*	**POP** eAX *58*	**POP** eCX *59*	**POP** eDX *5A*	**POP** eBX *5B*	**POP** eSP *5C*	**POP** eBP *5D*	**POP** eSI *5E*	**POP** eDI *5F*
PUSHA *60*	**POPA** *61*	**BOUND** Gv Ma *62*	**ARPL** Ew Gw *63*	**FS:** *64*	**GS:** *65*	**OPSIZE:** *66*	**ADSIZE:** *67*	**PUSH** Iv *68*	**IMUL** Gv Ev Iv *69*	**PUSH** Ib *6A*	**IMUL** Gv Ev Ib *6B*	**INSB** Yb DX *6C*	**INSW** Yz DX *6D*	**OUTSB** DX Xb *6E*	**OUTSW** DX Xv *6F*
JO Jb *70*	**JNO** Jb *71*	**JB** Jb *72*	**JNB** Jb *73*	**JZ** Jb *74*	**JNZ** Jb *75*	**JBE** Jb *76*	**JA** Jb *77*	**JS** Jb *78*	**JNS** Jb *79*	**JP** Jb *7A*	**JNP** Jb *7B*	**JL** Jb *7C*	**JNL** Jb *7D*	**JLE** Jb *7E*	**JNLE** Jb *7F*
ADD Eb Ib *80*	**ADD** Ev Iv *81*	**SUB** Eb Ib *82*	**SUB** Ev Ib *83*	**TEST** Eb Gb *84*	**TEST** Ev Gv *85*	**XCHG** Eb Gb *86*	**XCHG** Ev Gv *87*	**MOV** Eb Gb *88*	**MOV** Ev Gv *89*	**MOV** Gb Eb *8A*	**MOV** Gv Ev *8B*	**MOV** Ew Sw *8C*	**LEA** Gv M *8D*	**MOV** Sw Ew *8E*	**POP** Ev *8F*
NOP *90*	**XCHG** eAX eCX *91*	**XCHG** eAX eDX *92*	**XCHG** eAX eBX *93*	**XCHG** eAX eSP *94*	**XCHG** eAX eBP *95*	**XCHG** eAX eSI *96*	**XCHG** eAX eDI *97*	**CBW** *98*	**CWD** *99*	**CALL** Ap *9A*	**WAIT** *9B*	**PUSHF** Fv *9C*	**POPF** Fv *9D*	**SAHF** *9E*	**LAHF** *9F*

Пример

B0 03 04 04 B4 4C CD 21

Складывает 3 и 4

Классификация команд по адресности

- Трехадресные машины (современный RISC: ARM, RISC-V): ADD R1, A, B
- Двухадресные машины (классический x86): ADD A, B
- Одноадресные машины (ранние ЭВМ, простые микроконтроллеры): LOAD A, а потом ADD B
- Безадресные машины (стековая архитектура, JVM, WebAssembly): PUSH A, PUSH B, ADD

Режимы адресации

как процессор ищет данные

- Непосредственная например, ADD EAX, 5
- Регистровая ADD EAX, EBX)
- Прямая MOV EAX, [0x00401000]
- Косвенная регистровая MOV EAX, [EBX]
- Индексная и со смещением (Base + Offset) MOV EAX, [EBX + ECX*4 + 8]

Регистры процессора: сущность, назначение, типы.

Что такое регистры

В отличие от ОЗУ (конденсаторы DRAM) и кэш-памяти (ячейки SRAM на 6 транзисторах), регистр строится на синхронных D-триггерах (flip-flops). Один разряд регистра — это один триггер, который мгновенно меняет своё состояние по фронту тактового сигнала.

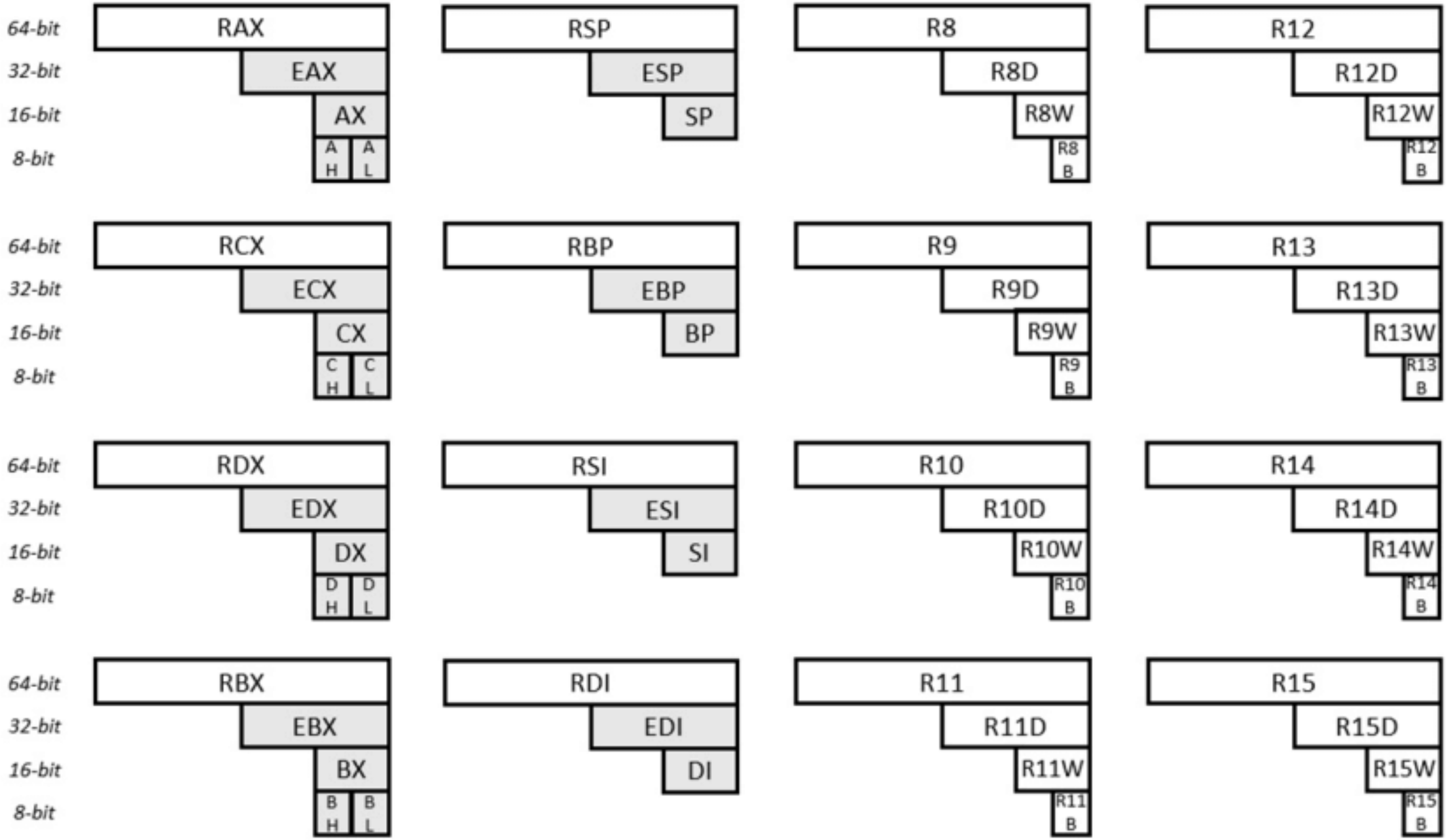
Доступ к регистру занимает менее 1 такта (доли наносекунды), тогда как обращение к ОЗУ требует сотни тактов

Разрядность определяет размер машинного слова, которое процессор может обработать за одну операцию

Классификация регистров по назначению

Регистры общего назначения

- EAX (Accumulator): для арифметических операций
- ECX (Counter): для хранения счетчика цикла
- EDX (Data): для арифметических операций и операций ввода-вывода
- EBX (Base): указатель на данные
- ESP (Stack pointer): указатель на верхушку стека
- EBP (Base pointer): указатель на базу стека внутри функции
- ESI (Source index): указатель на источник при операциях с массивом
- EDI (Destination index): указатель на место назначения в операциях с массивами
- EIP: указатель адреса следующей инструкции для выполнения
- EFLAGS: регистр флагов, содержит биты состояния процессора


64-bit
32-bit
16-bit
8-bit
RAX
EAX
AX
AH
AL
RSP
ESP
SP
R8
R8D
R8W
R8B
R12
R12D
R12W
R12B
RCX
ECX
CX
CH
CL
RBP
EBP
BP
R9
R9D
R9W
R9B
R13
R13D
R13W
R13B
RDX
EDX
DX
DH
DL
RSI
ESI
SI
R10
R10D
R10W
R10B
R14
R14D
R14W
R14B
RBX
EBX
BX
BH
BL
RDI
EDI
DI
R11
R11D
R11W
R11B
R15
R15D
R15W
R15B

Классификация регистров по назначению

Специальные и системные регистры

- **Указатель команд (RIP в x86 / PC в ARM).**
- **Указатель стека (RSP / SP)**
- **Регистр флагов (RFLAGS / CPSR)**
  - ZF (Zero Flag) — результат равен нулю.
  - CF (Carry Flag) — произошло беззнаковое переполнение (перенос из старшего бита).
  - OF (Overflow Flag) — переполнение при работе со знаковыми числами.
  - SF (Sign Flag) — результат отрицательный.
- **Управляющие регистры (CR0, CR3 в x86)**

Классификация регистров по назначению

Векторные регистры

Отдельный массив сверхширокоих регистров (XMM — 128 бит, YMM — 256 бит, ZMM — 512 бит). В один 512-битный регистр помещается сразу шестнадцать 32-битных чисел с плавающей точкой (float), что позволяет обрабатывать пиксели графики или матрицы нейросетей в 16 раз быстрее.

Параллелизм вычислений.

Проблема

До середины 2000-х годов производительность росла линейно за счёт уменьшения техпроцесса и повышения тактовой частоты. Но динамическая мощность процессора растёт пропорционально кубу частоты. Когда чипы начали выделять тепло на уровне утюга, единственным путём развития стал параллелизм.

Multi-Threading

